

宝宝巴适 口头汇报 Q&A 准备

一、电梯演讲（如果只给 30 秒介绍系统）

我们做了一个面向 6-12 月龄婴儿的辅食推荐与过敏预警系统。核心设计是「安全不过大模型」——规则引擎做安全决策，LLM 做排序和解释。规则引擎覆盖过敏原、月龄、质地、添加物、营养素等 9 个维度，知识底座包含 40 种食材、9 类过敏原、28 个 umbrella term。评测走了 22 个用例覆盖候选选择、安全决策、周计划生成、开放问答四类任务。

二、高频问题

Q1: 为什么不让大模型直接做安全决策？

直接答案：LLM 对过敏原映射、月龄阈值这类「必须 100% 准确」的任务不可靠——会漏判、会幻觉。过敏这件事一次误判就可能出问题。

具体例子：

- 牛奶蛋白过敏 → LLM 可能不认为 whey/酪蛋白也在过敏范围内。规则引擎通过 `allergens.json` 的 `aliases + cross_reactive` 确保全覆盖
- 配料表的乳清 (whey) 英文写，LLM 翻译可能丢，规则引擎用映射表直接命中
- 整颗葡萄的窒息风险：LLM 不知道 12 月龄以下不能吃整颗，规则引擎按月龄自动分级

我们的分层策略：

- 规则引擎：只说「能不能吃」
- LLM：说「怎么吃更好吃、更科学」

Q2: 规则引擎具体是怎么工作的？9 个模块各自做什么？

```
RuleEngine.evaluate_food(food, profile)
├─ 1.1 过敏原检测      → allergens.json 匹配 + 血缘扩展 (milk→whey/酪蛋白)
├─ 1.2 月龄门槛        → <6月龄统一 avoid, <min_age_months 判 caution
├─ 1.3 窒息风险分级    → <12月龄整颗/硬质→avoid, ≥12月龄→caution
├─ 1.4 质地匹配        → canonical 三等级 (puree/minced/chunky)，不匹配→caution
├─ 1.5 添加糖/盐       → <12月龄 avoid, ≥12月龄 caution
├─ 1.6 营养素覆盖      → 铁/锌/蛋白质等关键营养素是否覆盖
├─ 1.7 添加糖提示      → 外部食材 contains_added_sugar 标记
└─ 1.8 配料未知        → 外部食材+配料为空→caution
```

每条规则返回：{ tag: avoid|caution|suitable, reason: str, rule_name: str, source: str }

最终 overall_tag = 所有规则中的最高警告级别

关键细节：

- strict_mode：评测模式下月龄差≤1月判 caution 而非 avoid（避免误杀边界食材）

- 规则来源可追溯：每条规则标注 source (CDC/WHO/Codex)，不是黑箱

Q3: 知识底座怎么建的？40 种食材、9 类过敏原从哪来？

食材：

- 基础来源：CDC "When, What, and How to Introduce Solid Foods" + WHO 补充喂养指南
- 营养数据：USDA FoodData Central（每 100g 的铁、锌、蛋白质等）
- 商品级数据：Open Food Facts API（配料表、品牌、条码），但被封 IP，实际手动录了一批

过敏原：

- 9 类核心：鸡蛋、牛奶、小麦、大豆、花生、坚果、鱼、虾蟹、芝麻
- 28 个 umbrella term：每个过敏原配 aliases（中/英/成分名）+ cross_reactive
- 示例：牛奶 → ["milk", "dairy", "whey", "casein", "lactose", "乳清", "酪蛋白"]
- 血缘扩展在规则引擎中自动完成

月龄阶段：

- 自研三阶段：吞咽期(6-8月)→蠕嚼期(9-10月)→细嚼期(11-12月)
- 每阶段定义 texture、meals_per_day、key_nutrients、禁止食物

Q4: LLM 在系统里具体用在哪些环节？

环节	LLM 角色	不经过 LLM 的
标签解析	从配料表图片 OCR 结果中提取实体（小麦、鸡蛋、添加剂）	-
用户画像	从自由文本提取过敏原、月龄、喂养方式	规则引擎的年龄计算
计划生成（LLM 模式）	接收 filtered safe foods + profile，生成 7 天多样计划	安全过滤（rule engine）
Chat 对话	RAG 检索 + LLM 自由回答	知识库查询（工具调用走结构化查询）
语义解析	开放文本食材名解析为 KB ID（如"iron fortified cereal" → 强化铁米粉）	KB 精确匹配优先

关键边界：LLM 生成的计划仍会后校验（ValidationAgent），二次过规则引擎。LLM 有自由但不出圈。

Q5: Neo4j 知识图谱用在哪？为什么加了这个？

用途：

- 食材之间的替代关系（猪肝 vs 牛肉 for 铁）
- 食材→营养素的多对多关系
- 过敏原→食材的「含有」关系
- 食材→月龄阶段的适配关系

为什么加：技术加分项。项目要求中「高阶要求」提到「进行现有AI模块的适配或自行开发AI模块」。58 条关系可查、可演示图查询（MATCH (f:Food)-[:RICH_IN]->(n:Nutrient)）。

现实：评测路径用 JSON backend（更快、对安全规则来说确定性更强），Neo4j 是展示用的。

Q6: 评测怎么做的？为什么选了这样的评测方式？

架构：苏同学维护独立评测框架，通过 `babybites_app_root` 引入我们的代码。做到评测代码和业务代码解耦。

四类任务：

- 1. candidate_selection：20 个候选食材 → 判 safe/avoid/caution + 规则ID
- 2. meal_plan_generation：给定池 → 生成 7 天计划
- 3. rag_vs_plain_gpt：开放文本回答质量
- 4. 安全决策：盲评 + Judge (Qwen) 评分

关键结果：

- 候选选择：安全准确率较高（过敏原/窒息风险基本覆盖）
- RAG 开放回答：8/10 胜 baseline
- 周计划（优化后）：LLM计划生成已持平 baseline
- 我们做了 5 轮修复迭代，每轮都是评测驱动

Q7: "安全不过大模型"这个设计理念——你们有没有做过消融实验验证它？

在回答时可以说：

- 纯 LLM baseline（评测中的 DeepSeek baseline）在候选选择上对过敏原覆盖不全——比如 whey 不算牛奶过敏、加糖果泥袋判 safe
- 我们的规则引擎补上了这些缺口
- 但规则引擎也有代价：过于保守（未知食材全判 caution），导致计划池偏小
- 未来会做正式的 A/B 消融实验（规则 only vs LLM only vs 规则+LLM）放在最终报告里

Q8: 系统有什么局限性？

局限	说明
知识库规模小	40 种食材 vs 现实数百种辅食，且缺少商品级数据（OFF被封）
规则保守	未知食材全判 caution，导致计划池偏小
计划多样性	规则模式下拉轮播，LLM模式有改善但仍依赖 prompt 质量
无个性化	未考虑预算、地域、口味偏好、文化习俗
离线运行	没有持续从用户反馈中学习的闭环（反馈存了但没做在线学习）
中文资源少	大部分标准是英文，中文家庭使用存在语言适配问题

Q9: 如果继续做，接下来会做什么？

- 1. **消融实验：**规则 only vs LLM only vs 规则+LLM 的正式对比
- 2. **计划个性化：**引入预算约束、地域食材偏好、口味偏好
- 3. **知识库扩充：**爬取更多商品数据（需要 VPN）、引入更多食材
- 4. **反馈闭环：**用户反馈自动调整后续计划推荐

5. 质地发展跟踪：根据月龄自动建议质地升级时间点

三、可能被追问的技术细节

T1: allergies.json 的 aliases 匹配逻辑

```
# 匹配逻辑（简化）
def match_allergen(ingredient_text, profile_allergies):
    for allergy_id in profile_allergies:
        allergen = allergens[allergy_id]
        for alias in allergen["aliases"]:
            if alias.lower() in ingredient_text.lower():
                return allergy_id # 命中
    return None
```

关键点：大小写不敏感、子串匹配、中英文都覆盖。

T2: canonical 质地三级归一化

```
# 所有中英文质地值归一化到三个等级
PUREE = {"puree", "purée", "泥糊状", "泥状", "糊状", "mashed"}
MINCED = {"minced", "碎末状", "细末", "minced", "ground"}
CHUNKY = {"chunky", "finger_food", "软块", "手指食物", "soft", "diced"}

# 比较：食材.质地_level <= 阶段.质地_level 则通过
```

这是评测中发现的问题是最大 bug 之一——之前食材用英文 "puree"，阶段用中文 "泥糊状"，永远不匹配，全部误判为质地不匹配。

T3: external_food 包装逻辑

```
def resolve_food(candidate, kb):
    # 1. 尝试精确匹配 KB (by name_zh, name_en, food_id)
    food = kb.get_food_by_name(candidate["food_name_zh"])
    if food: return food

    # 2. 未匹配 → 包装为 external food
    return {
        "_external": True,
        "name_zh": candidate["food_name_zh"],
        "category": candidate.get("category", "unknown"),
        "_ingredient_text": candidate.get("ingredient_text", ""),
        "_contains_added_sugar": candidate.get("contains_added_sugar", False),
        "_contains_added_salt": candidate.get("contains_added_salt", False),
        "_is_choking_risk": candidate.get("is_choking_risk_candidate", False),
        "_sodium_mg": candidate.get("sodium_mg_per_100g"),
        "_allergen_flags": {k: candidate.get(f"contains_{k}", False) for k in
```

```
[...]],  
    ...  
}
```

这对评测很重要——评测用例中有大约 1/3 的食材不在 KB 里，外部包装让它们也能走规则引擎。

T4: 规则ID映射不是子串匹配了

最初是子串匹配，比如 reason 里有「过敏」就映射 `R_ALLERGY_KNOWN`，但随便一个 reason 都可能有「过敏」两个字。现在改成精确映射：

- reason 中必须出现关键词且该 reason 的语义符合该规则ID
- 每个 reason 只映射一个规则ID

```
patterns = [  
    ("添加糖", "R_ADDED_SUGAR_CAUTION"),      # 精确匹配  
    ("牛奶", "R_ALLERGY_MILK"),                # 不是泛化的"过敏"  
    ("配料未知", "R_INSUFFICIENT_INGREDIENT_INFO"),  
    ...  
]  
for reason in reasons:  
    for keyword, rule_id in patterns:  
        if keyword in reason:  
            ids.append(rule_id)  
            break # 每个 reason 只匹配一次
```

T5: LLM 计划生成的 prompt 怎么设计的

核心设计：LLM 只能从规则引擎过滤后的安全食材池中选，不能加新食材。prompt 三要素：

1. 约束：只能用提供的食材
2. 信息：完整画像（月龄/过敏原/已尝试/阶段）
3. 指令：新食材优先周一/同类不连续/一周覆盖多类别

LLM 失败自动回退规则模式（fallback）。

T6: FeedbackStore 的实现

```
# 存储在 SQLite  
# 读取: get_food_safety_label(food_name) → safe|caution|avoid  
# 写入: record_feedback(baby_id, food_name, reaction, date)  
  
# 判断逻辑  
if reaction in ["腹泻", "呕吐", "皮疹", "呼吸困难"]:  
    label = "avoid"  
elif reaction in ["轻微不适", "不喜欢"]:  
    label = "caution"
```

四、汇报故事线建议（6分钟）

0:00-0:30 场景与问题：6-12月龄辅食添加，家长信息过载、担心过敏
0:30-1:30 系统概述：输入→输出，展示Demo视频/截图
1:30-2:30 技术架构：规则+LLM分层，为什么安全不过大模型
2:30-3:30 核心难点：9规则模块、知识底座构建、外部食材包装
3:30-4:30 评测结果：四类任务对比、关键修复案例
4:30-5:00 系统局限与边界
5:00-5:30 后续计划（消融实验、个性化）
5:30-6:00 开放问答预留